

Haneen Alaa Hassan

Day3 c#

Essay questions

Q1 – Difference between int.Parse() and Convert.ToInt32()

Both of the methods `int.Parse()` and `Convert.ToInt32()` can convert a string into an integer, but their behavior differs when the argument passed to them is null.

`int.Parse(null)` will throw `ArgumentNullException` since it expects a valid string with a number in it.

`Convert.ToInt32(null)`, on the other hand, will return 0.

Example:

```
int.Parse(null); // Will throw ArgumentNullException
```

```
Convert.ToInt32(null); // Will return 0
```

That means that `Convert.ToInt32()` will work with null arguments differently from `int.Parse()`.

Q2 – Why is TryParse() Better than Parse()?

`TryParse()` method is better to use in user-facing applications since it will not throw an exception when the user entered incorrect data.

`Parse()` will throw an exception if the entered string cannot be converted to an integer. For that reason, exception handling (try-catch) should be used.

`TryParse()`, on the other hand, will return a Boolean value:

true if the conversion was successful,
false if the conversion failed.

Example:

```
if (int.TryParse(input, out int number))  
{  
    Console.WriteLine(number);  
}  
else  
{  
    Console.WriteLine("Invalid input.");  
}
```

Thus, TryParse() is more safe and more efficient way to validate the input.

Q3 – What Is the Purpose of GetHashCode()?

GetHashCode() returns an integer that represents the hash code of an object.

The main purposes of GetHashCode() include supporting hash-based collections like:

Dictionary<TKey, TValue>

HashSet<T>

Hash codes allow these collections to find objects easily.

If two objects are equal, they must return the same hash code. However, different objects can occasionally have the same hash code; this is called hash collision.

Thus, GetHashCode() is mostly used for fast lookup and comparisons in hash-based collections.

Q4 – The Significance of Reference Equality in .NET

Reference equality determines whether two references point to the same object in the memory.

In C# ReferenceEquals() can be used for this.

Example:

```
MyClass obj1 = new MyClass();
```

```
MyClass obj2 = obj1;
```

```
Console.WriteLine(ReferenceEquals(obj1, obj2));
```

The output will be true since both references point to the same object.

Reference equality is useful while working with reference types since modifying the object via one reference may become visible through another reference pointing to the same object.

Q5 – Why Is string Immutable in C#?

string is immutable in C#, meaning that once the string object is created, its contents cannot be modified.

While it seems that we modify the string, C# actually creates a new string object.

Example:

```
string text = "Hello";  
text = text + " World";
```

The original

StringBuilder uses a mutable internal buffer, allowing operations such as Append(), Insert(), Remove(), and Replace() without creating a new StringBuilder object each time.

Therefore, StringBuilder is more efficient when many modifications are required.

Q8 — Which String Formatting Method is Most Used and Why?

String interpolation is one of the most commonly used formatting methods in modern C# because it is simple, readable, and easy to maintain.

Example:

```
string name = "Haneen";  
int age = 21;
```

```
Console.WriteLine($"My name is {name} and I am {age} years  
old.");
```

Compared with string.Format(), interpolation makes it easier to see the values directly inside the string.

Therefore, string interpolation is usually preferred for normal string formatting in modern C# code.

Q9 — How is StringBuilder Designed for Frequent Modifications?

StringBuilder is designed to handle frequent modifications by using a mutable internal buffer.

Unlike a string, which cannot be changed after creation, a StringBuilder can be modified directly.

It provides methods such as:

Append()

Insert()

Remove()

Replace()

This design reduces unnecessary string allocations and makes it suitable for building or modifying large strings repeatedly.

Part 02

Q11 — What is Enum?

An enum, short for enumeration, is a value type in C# used to define a set of named constant values.

Enums are useful when a variable should have one value from a limited and predefined set of options.

For example:

```
enum Days
```

```
{
```

```
    Saturday,
```

```
    Sunday,
```

```
    Monday,
```

```
Tuesday
```

```
}
```

We can then use it like this:

```
Days today = Days. Sunday;
```

Enums improve code readability because they allow us to use meaningful names instead of numbers.

When are Enums Used?

Enums are commonly used when there is a fixed set of possible values, such as:

Days of the week.

Months.

Application states.

User roles.

Directions.

File or processing states.

Three Common Built-in Enums

Three commonly used built-in enums in .NET are:

DayOfWeek

StringComparison

ConsoleColor

For example:

```
DayOfWeek day = DayOfWeek.Monday;
```

Enums make code easier to understand and reduce the possibility of using invalid values.

Q12 — String vs StringBuilder

Both string and StringBuilder are used to work with text, but they are designed for different situations.

String

A string is immutable, which means its contents cannot be changed after the object is created.

It is the better choice when:

The text does not change frequently.

You are working with small amounts of text.

You need simple string operations.

You want straightforward and readable code.

Example:

```
string name = "Haneen";
```

StringBuilder

StringBuilder is mutable and is designed for frequent modifications.

It is a better choice when:

You perform many concatenations.

You modify large strings frequently.

You use operations such as Append(), Insert(), Remove(), and Replace().

Performance and memory allocations are important.

Example:

```
StringBuilder sb = new StringBuilder();  
sb.Append("Hello");  
sb.Append(" World");
```

Conclusion

For normal text operations, string is usually the best and simplest choice.

For frequent and large-scale modifications, StringBuilder is usually more efficient.

Part 03 — Bonus

Q14 — What is a User-Defined Constructor?

A constructor is a special method in a class that is automatically called when an object is created.

A user-defined constructor is a constructor that is explicitly written by the programmer to initialize an object.

For example:

```
class Student  
{  
    public string Name;  
    public int Age;  
    public Student(string name, int age)  
    {
```



```
Name = name;
```

```
Age = age;
```

```
}
```

```
}
```

We can create an object using the constructor:

```
Student student = new Student("Haneen", 21);
```

The constructor initializes the object's fields when the object is created.

Role of a Constructor

Constructors are useful because they:

- Initialize object data.

- Ensure an object starts with valid values.

- Make object creation easier.

- Allow required values to be provided when creating an object.

Therefore, a user-defined constructor gives the programmer control over how an object's initial state is created.